第三章 分组密码与高级数据加密标准

杨礼珍

同济大学计算机科学与技术系

Outline

1. Introduction
2. Subtitution-permutation network
3. Linear Cryptanalysis
4. difference cryptanalysis
5. DES
6. AES
7. work mode

本章作业

本章作业以这里列出的为准。
第三章习题3.1，3.2，3.3，3.7
习题3.1：设y是算法3.1在输入为x时的输出，π_S与π_P与例3.1中的定义相同。换言之

$$y = \mathrm{SPN}(x, \pi_S, \pi_P, (K^1, \ldots, K^{N_r+1}))$$

这里$(K^1, \ldots, K^{N_r+1})$是密码编排方案。试找出一个代换π_{S^*}和一个置换π_{P^*}，满足

$$x = \mathrm{SPN}(y, \pi_{S^*}, \pi_{P^*}, (L^{N_r+1}, \ldots, L^1))$$

其中，L^i是K^i的置换。

本章作业

习题3.2：证明解密一个Feistel密码相同于对密文使用加密算法，但密钥编排方案要逆序使用。

习题3.3：设$\mathrm{DES}(x, K)$表示使用DES在密钥K下对明文x进行加密，假定$y = \mathrm{DES}(x, K)$，$y' = \mathrm{DES}(c(x), c(K))$，这里$c(\cdot)$表示对其自变量按比特位取反。试证明$y' = c(y)$（即如果把明文和密钥都按比特位取反，则密文同样是按比特反取反）。注意证明这一点只需要使用DES的总体描述—与S盒的实际结构和系统的其它部件无关。

习题3.7：设明文分组序$x_1, \ldots, x_n$产生的密文分组序列为$y_1, \ldots, y_n$。假设一个密文分组y_i在传输时出现了错误（即某些1变成了0，或者相反）。证明不能正确解密的明文分组数目在应用ECB或OFB模式时为1；在应用CBC或CFB模式时为2。

现代密码的加密方式

- **对称加密体制**—容易从加密密钥计算出解密密钥，一般加密密钥和解密密钥一致，加密密钥和解密密钥都需要保密，因此也称为私钥密码体制。
  - **流密码**：由种子密钥产生密钥流，密钥流与明文流作用产生密文流，在第1.1.7节简单介绍过。
  - **分组密码**：明文以分组为单位进行加密，当今绝大部分分组密码都是乘积密码（见上一节），本章内容。
- **非对称密码体制**—从加密密钥计算出解密密钥是困难的，加密密钥是公开的，而解密密钥是保密的，因此也称为公钥密码体制，将在第五、六章介绍。

现代密码的加密方式

- **对称加密体制**—容易从加密密钥计算出解密密钥，一般加密密钥和解密密钥一致，加密密钥和解密密钥都需要保密，因此也称为私钥密码体制。
  - **流密码：**由种子密钥产生密钥流，密钥流与明文流作用产生密文流，在第1.1.7节简单介绍过。
  - **分组密码：**明文以分组为单位进行加密，当今绝大部分分组密码都是乘积密码（见上一节），本章内容。
- **非对称密码体制**—从加密密钥计算出解密密钥是困难的，加密密钥是公开的，而解密密钥是保密的，因此也称为公钥密码体制，将在第五、六章介绍。

现代密码的加密方式

- **对称加密体制**—容易从加密密钥计算出解密密钥，一般加密密钥和解密密钥一致，加密密钥和解密密钥都需要保密，因此也称为私钥密码体制。
  - **流密码：**由种子密钥产生密钥流，密钥流与明文流作用产生密文流，在第1.1.7节简单介绍过。
  - **分组密码：**明文以分组为单位进行加密，当今绝大部分分组密码都是乘积密码（见上一节），本章内容。
- **非对称密码体制**—从加密密钥计算出解密密钥是困难的，加密密钥是公开的，而解密密钥是保密的，因此也称为公钥密码体制，将在第五、六章介绍。

现代密码的加密方式

- **对称加密体制**—容易从加密密钥计算出解密密钥，一般加密密钥和解密密钥一致，加密密钥和解密密钥都需要保密，因此也称为私钥密码体制。
  - **流密码：**由种子密钥产生密钥流，密钥流与明文流作用产生密文流，在第1.1.7节简单介绍过。
  - **分组密码：**明文以分组为单位进行加密，当今绝大部分分组密码都是乘积密码（见上一节），本章内容。
- **非对称密码体制**—从加密密钥计算出解密密钥是困难的，加密密钥是公开的，而解密密钥是保密的，因此也称为公钥密码体制，将在第五、六章介绍。

分组密码的基本术语

分组长度 明文分组长度（长度=比特数），和密文分组长度一样

种子密钥 分组密码的密钥也称为种子密钥，以区别于轮密钥。

密钥长度 种子密钥的长度，不一定等于分组长度

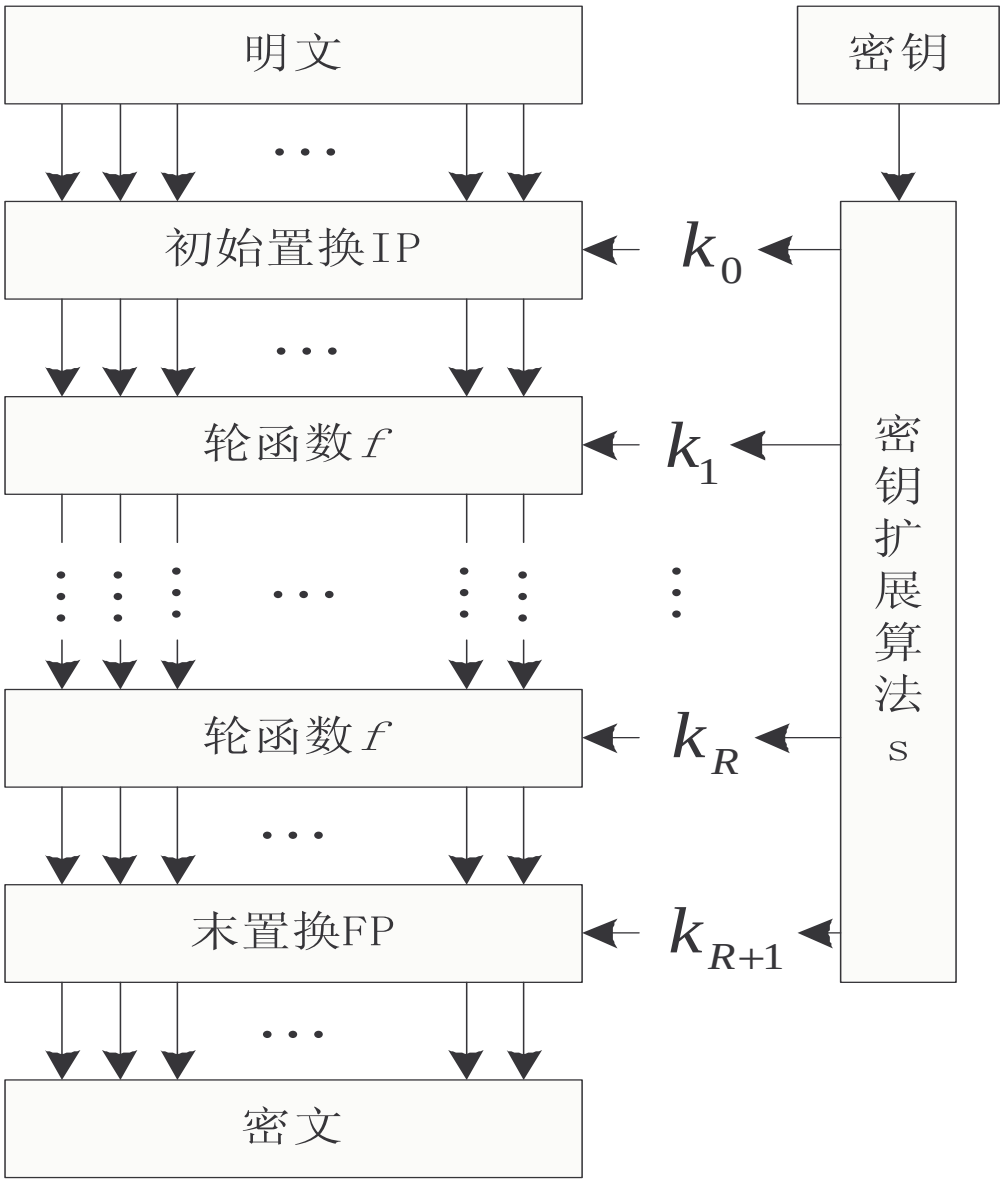
轮函数

轮密钥 轮密钥的长度不一定等于密钥长度

密钥编排算法(方案)，或密码扩展算法(方案) 种子密钥由密钥编排方案扩展成若干个轮密钥，作为轮函数的输入。

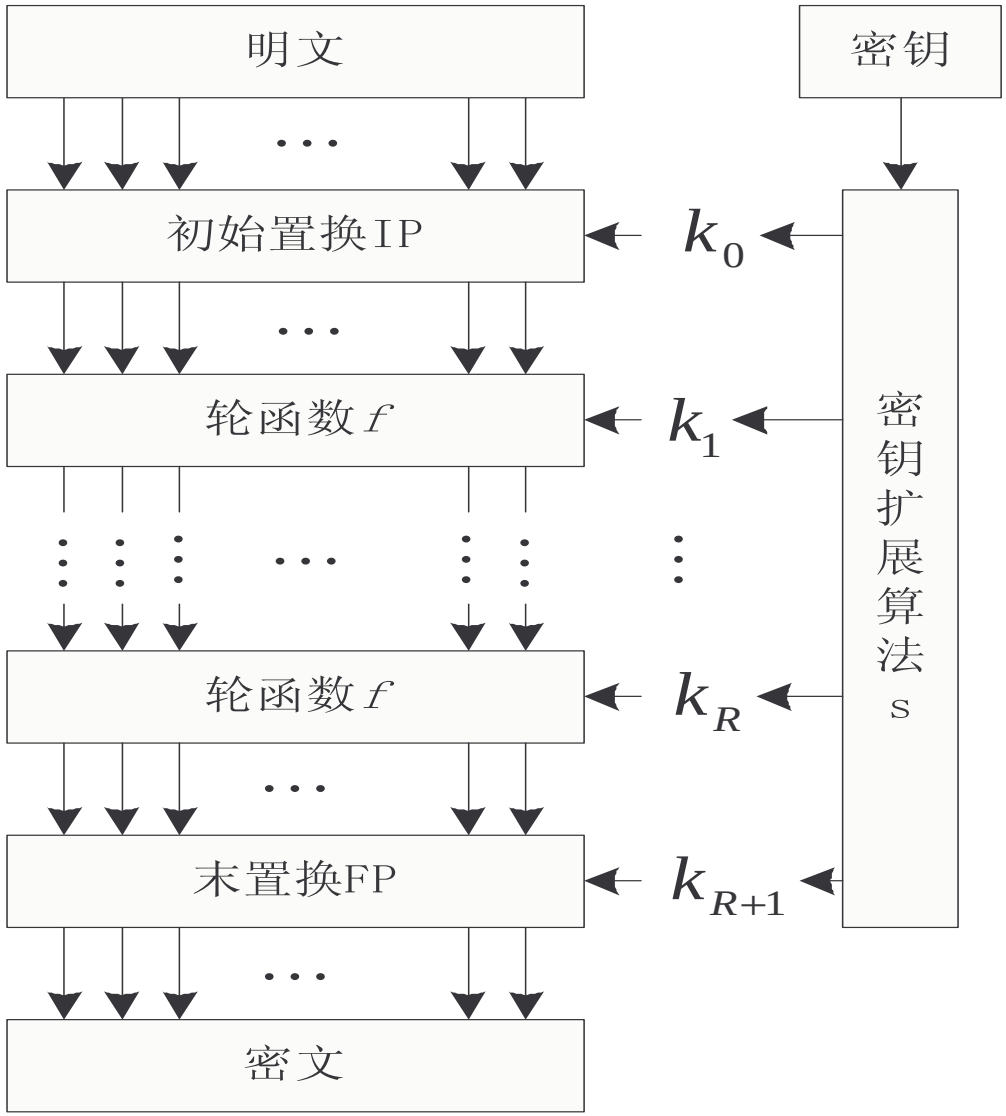
迭代次数

分组密码的一般形式



分组密码的一般形式

解密函数：加密函数的逆函数，因此解密函数的运算过程和加密函数相反，轮函数是加密函数的轮函数的逆，轮密钥与加密函数的轮密钥顺序相反。



分组密码的一般形式

备注：以上给出的是分组密码的一般形式，不是每个分组加密都严格遵循以上一般形式。

本章内容

3.2节 代换-置换网络
3.3节 线性密码分析（简要了解）
3.4节 差分密码分析（简要了解）
3.5节 数据加密标准（DES）(不需要记忆算法细节)
3.6节 高级数据加密标准（AES）(不需要记忆算法细节)
3.7节 分组密码的工作模式

迭代密码的两种类型

根据轮函数的形式，分为两种类型：

- Feistel型密码：代表密码是DES（将在第3.5节介绍）
- 代换-置换网络(SPN)：代表密码是AES（将在第3.6节介绍）

迭代密码的两种类型

根据轮函数的形式，分为两种类型：

- Feistel型密码：代表密码是DES（将在第3.5节介绍）
- 代换-置换网络(SPN)：代表密码是AES（将在第3.6节介绍）

代换-置换网络的一般形式

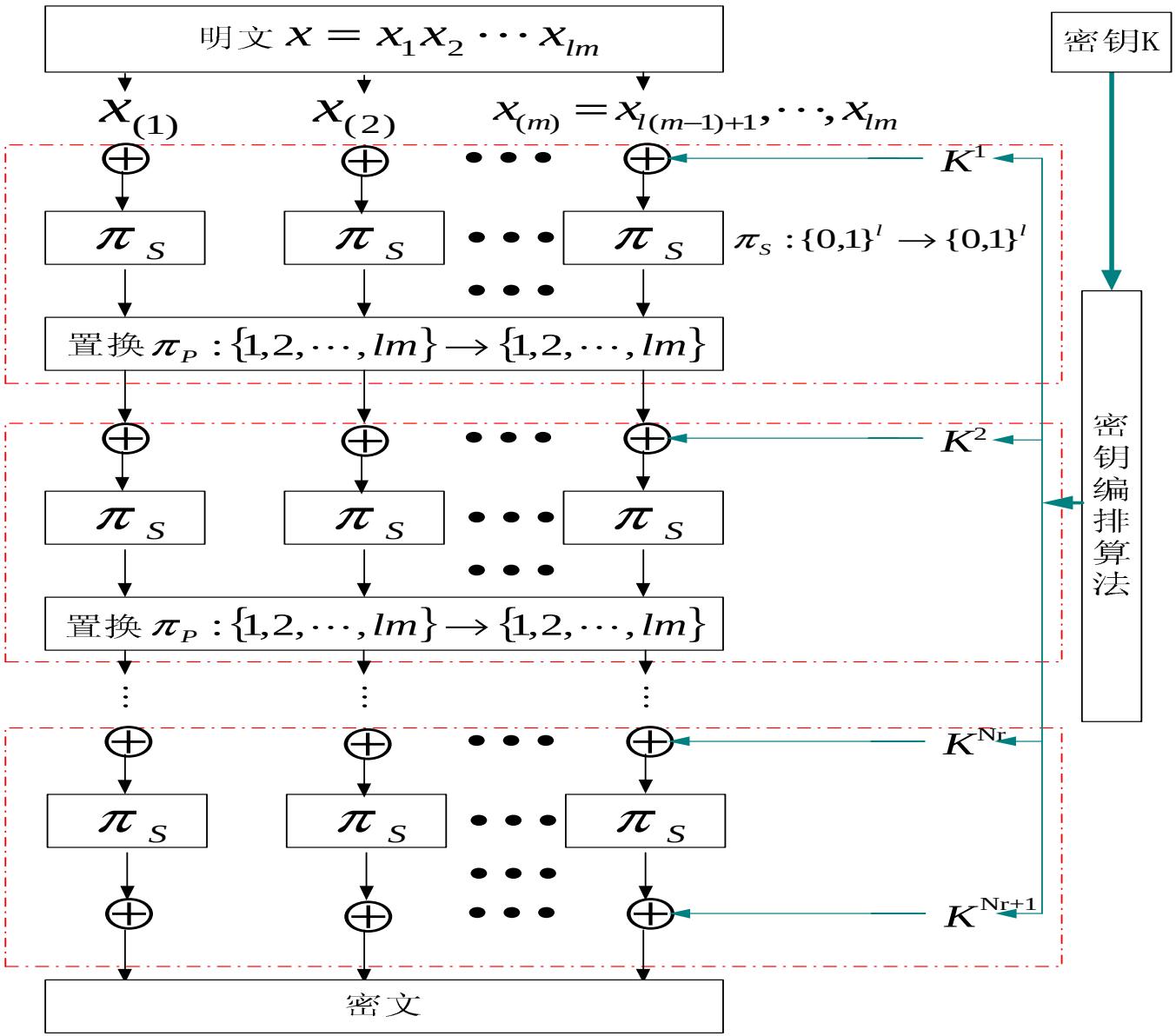
代换-置换网络来自Shannon的扩散-混淆的设计思想：

- 扩散：将明文中一个比特的影响扩散到密文中很多比特，从. 而将明文的统计结构隐藏起来。π_P起到扩散作用。
- 混淆：采用数据变换，使密文的统计特性和明文的统计特性之间的关系更为复杂。π_S起到混淆作用。

代换-置换网络的一般形式

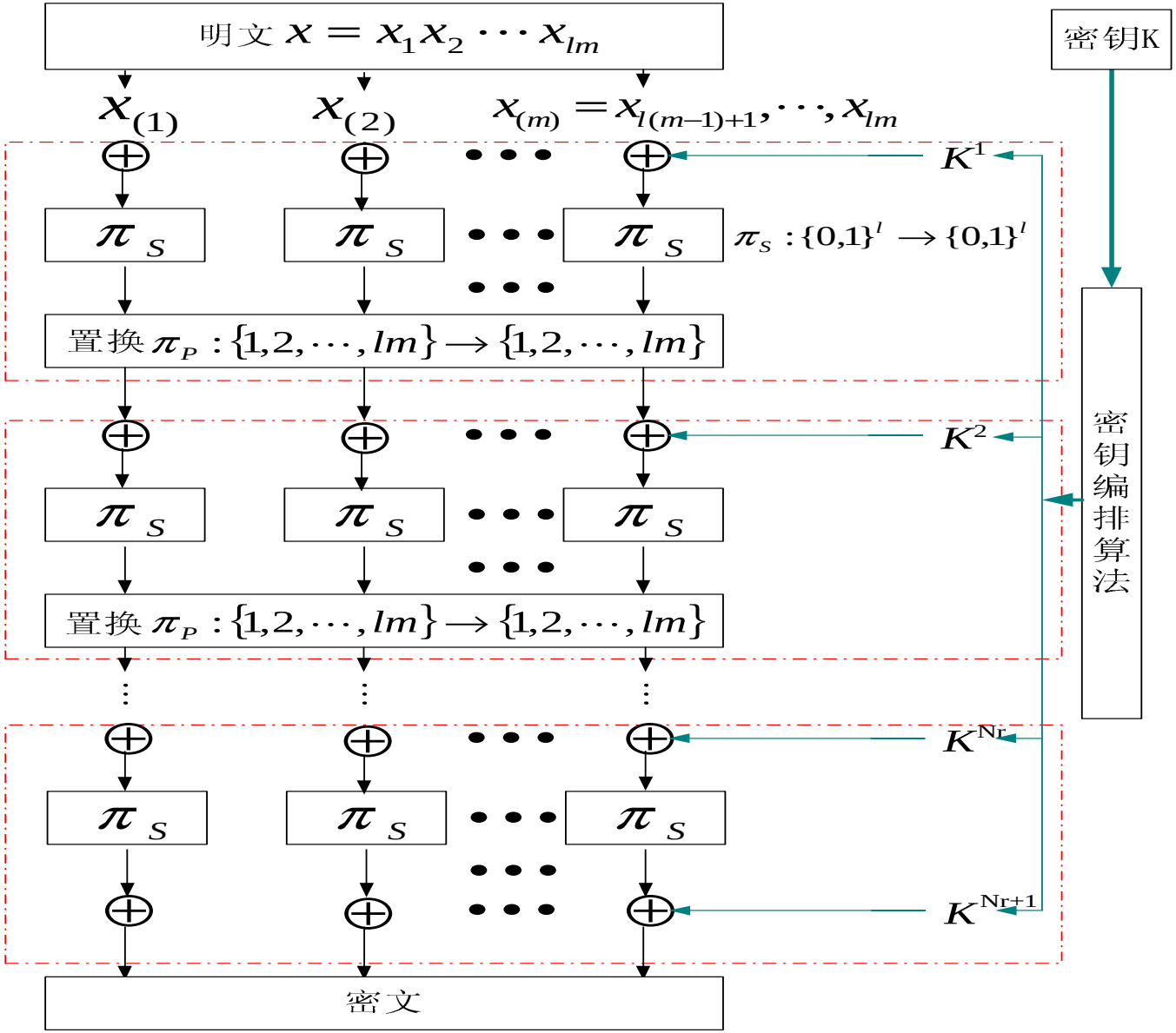
π_S：$\{0,1\}^l$自身上的双射，非线性运算，代换层，S盒。

π_P：$\{1,2,\ldots,lm\}$上的置换，线性运算，置换层。



代换-置换网络的一般形式

解密函数：轮函数顺序与加密函数相反，过程相反，S盒是π_S的逆函数，置换是π_P的逆函数。思考练习3.1。



例3.1

参数定义： $l = m = 4 = N_r = 4$

S盒π_S定义：输入z和输出$\pi_S(z)$都以十六进制表示，即

$$0 \leftrightarrow \{0,0,0,0\}, 1 \leftrightarrow \{0,0,0,1\}, \cdots, 9 \leftrightarrow \{1,0,0,1\}$$
$$A \leftrightarrow \{1,0,1,0\}, \cdots F \leftrightarrow \{1,1,1,1\}$$

z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\pi_S(z)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

置换π_P定义：

$$\pi_P(i+4j) = 4i + j - 3 \quad \pi_P(16) = 16, \quad 0 \le j \le 3$$

z	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\pi_P(z)$	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

密钥编排算法定义：

$$32\text{比特密钥}K = K_1||K_2||K_3||K_4||K_5||K_6||K_7||K_8$$

轮密钥：

$$K^1 = K_1||K_2||K_3||K_4 \quad K^2 = K_2||K_3||K_4||K_5 \quad K^3 = K_3||K_4||K_5||K_6$$
$$K^4 = K_4||K_5||K_6||K_7 \quad K^5 = K_5||K_6||K_7||K_8$$

例3.1

参数定义： $l = m = 4 = N_r = 4$

S盒π_S定义：输入z和输出$\pi_S(z)$都以十六进制表示，即

$$0 \leftrightarrow \{0,0,0,0\}, 1 \leftrightarrow \{0,0,0,1\}, \cdots, 9 \leftrightarrow \{1,0,0,1\}$$
$$A \leftrightarrow \{1,0,1,0\}, \cdots F \leftrightarrow \{1,1,1,1\}$$

z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\pi_S(z)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

置换π_P定义：

$$\pi_P(i+4j) = 4i + j - 3 \quad \pi_P(16) = 16, \quad 0 \leq j \leq 3$$

z	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\pi_P(z)$	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

密钥编排算法定义：

$$32\text{比特密钥}K = K_1||K_2||K_3||K_4||K_5||K_6||K_7||K_8$$

轮密钥：

$$K^1 = K_1||K_2||K_3||K_4 \quad K^2 = K_2||K_3||K_4||K_5 \quad K^3 = K_3||K_4||K_5||K_6$$
$$K^4 = K_4||K_5||K_6||K_7 \quad K^5 = K_5||K_6||K_7||K_8$$

例3.1

参数定义： $l = m = 4 = N_r = 4$

S盒π_S定义：输入z和输出$\pi_S(z)$都以十六进制表示，即

$$0 \leftrightarrow \{0,0,0,0\}, 1 \leftrightarrow \{0,0,0,1\}, \cdots, 9 \leftrightarrow \{1,0,0,1\}$$
$$A \leftrightarrow \{1,0,1,0\}, \cdots F \leftrightarrow \{1,1,1,1\}$$

z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\pi_S(z)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

置换π_P定义：

$$\pi_P(i+4j) = 4i + j - 3 \quad \pi_P(16) = 16, \quad 0 \le j \le 3$$

z	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\pi_P(z)$	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

密钥编排算法定义：

$$32\text{比特密钥}K = K_1||K_2||K_3||K_4||K_5||K_6||K_7||K_8$$

轮密钥：

$$K^1 = K_1||K_2||K_3||K_4 \quad K^2 = K_2||K_3||K_4||K_5 \quad K^3 = K_3||K_4||K_5||K_6$$
$$K^4 = K_4||K_5||K_6||K_7 \quad K^5 = K_5||K_6||K_7||K_8$$

例3.1

参数定义： $l = m = 4 = N_r = 4$

S盒π_S定义：输入z和输出$\pi_S(z)$都以十六进制表示，即

$$0 \leftrightarrow \{0,0,0,0\}, 1 \leftrightarrow \{0,0,0,1\}, \cdots, 9 \leftrightarrow \{1,0,0,1\}$$
$$A \leftrightarrow \{1,0,1,0\}, \cdots F \leftrightarrow \{1,1,1,1\}$$

z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\pi_S(z)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

置换π_P定义：

$$\pi_P(i+4j) = 4i + j - 3 \quad \pi_P(16) = 16, \quad 0 \leq j \leq 3$$

z	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\pi_P(z)$	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

密钥编排算法定义：

$$32\text{比特密钥}K = K_1||K_2||K_3||K_4||K_5||K_6||K_7||K_8$$

轮密钥：

$$K^1 = K_1||K_2||K_3||K_4 \quad K^2 = K_2||K_3||K_4||K_5 \quad K^3 = K_3||K_4||K_5||K_6$$
$$K^4 = K_4||K_5||K_6||K_7 \quad K^5 = K_5||K_6||K_7||K_8$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

例3.1

输入：明文：$x = 0010\ 0110\ 1011\ 0111$

密钥：$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

加密过程：

$$
\begin{array}{lllll}
x & & = & 0010\ 0110\ 1011\ 0111 & \\
K^1 & & = & 0011\ 1010\ 1001\ 0100 & \\
u^1 & = x \oplus K^1 & = & 0001\ 1100\ 0010\ 0011 & = 1\ C\ 2\ 3 \\
v^1 & = \pi_S(u^1) & = & 0100\ 0101\ 1101\ 0001 & = 4\ 5\ D\ 1 \\
w^1 & = \pi_P(v^1) & = & 0010\ 1110\ 0000\ 0111 & \\
 & & \vdots & & \\
w^3 & & = & 1110\ 0100\ 0110\ 1110 & \\
K^4 & & = & 0100\ 1101\ 0110\ 0011 & \\
u^4 & = w^3 \oplus K^4 & = & 1010\ 1001\ 0000\ 1101 & = A\ 9\ 0\ B \\
v^4 & = \pi_S(u^4) & = & 0110\ 1010\ 1110\ 1001 & = 6\ A\ E\ C \\
K^5 & & = & 1101\ 0110\ 0011\ 1111 & \\
y & = v^4 \oplus K^5 & = & 1011\ 1100\ 1101\ 0110 &
\end{array}
$$

代换-置换网络特点和变体

SPN的特点是简单有效：

- S盒π_S可以以查表方式实现，$\pi_S : \{0,1\}^l \to \{0,1\}^l$所需的存储空间是$l2^l$。
- S盒π_S的作用是把明文和密钥局部混淆，是非线性运算，π_P起全局扩散作用，是线性运算。

代换-置换网络特点和变体

SPN的特点是简单有效：

- S盒π_S可以以查表方式实现，$\pi_S : \{0,1\}^l \to \{0,1\}^l$所需的存储空间是$l2^l$。
- S盒π_S的作用是把明文和密钥局部混淆，是非线性运算，π_P起全局扩散作用，是线性运算。

代换-置换网络特点和变体

SPN的特点是简单有效：

- S盒π_S可以以查表方式实现，$\pi_S : \{0,1\}^l \to \{0,1\}^l$所需的存储空间是$l2^l$。
- S盒π_S的作用是把明文和密钥局部混淆，是非线性运算，π_P起全局扩散作用，是线性运算。

SPN的变体：

- 使用多个S盒π_S，如AES中使用了8个不同的S盒。
- 每一轮中包含一个可逆的线性运算，该线性变换要么代替置换π_P，要么作为π_P的补充。

代换-置换网络特点和变体

SPN的特点是简单有效：

- S盒π_S可以以查表方式实现，$\pi_S : \{0,1\}^l \to \{0,1\}^l$所需的存储空间是$l2^l$。
- S盒π_S的作用是把明文和密钥局部混淆，是非线性运算，π_P起全局扩散作用，是线性运算。

SPN的变体：

- 使用多个S盒π_S，如AES中使用了8个不同的S盒。
- 每一轮中包含一个可逆的线性运算，该线性变换要么代替置换π_P，要么作为π_P的补充。

3.2节的作业

练习3.1

符号说明：

- $X[i_1, i_2, \ldots, i_d]$表示比特向量X的第$i_1, i_2, \ldots, i_d$比特的异或，即

$$X[i_1, i_2, \ldots, i_d] = X[i_1] \oplus X[i_2] \oplus \ldots X[i_d]$$

- X表示分组密码的明文输入，$W^i(i = 1, \ldots, N_r + 1)$表示第$i$轮输出，或第$i + 1$轮输入，那么密文$Y = W^{N^r+1}$。

如果已知下式成立的概率为p

$$X[i_1, \ldots, i_a] \oplus Y[z_1, \ldots, z_f] = K[j_1, \ldots, j_b] \tag{1}$$

若$p \neq 1/2$，有以下线性攻击：

输入 N对明文—密文对

1. 令T为使得(1)的左边$= 0$的明文—密文对的数量。
2. 当$p > 1/2$时，
   - 若$T > N/2$，则猜测$K[j_1, \ldots, j_b] = 0$
   - 若$T < N/2$，则猜测$K[j_1, \ldots, j_b] = 1$
3. 当$p < 1/2$时，
   - 若$T > N/2$，则猜测$K[j_1, \ldots, j_b] = 1$
   - 若$T < N/2$，则猜测$K[j_1, \ldots, j_b] = 0$

3.3 线性密码分析原理

N	$\frac{1}{4}\|p-1/2\|^{-2}$	$\frac{1}{2}\|p-1/2\|^{-2}$	$\|p-1/2\|^{-2}$
成功概率	84.1%	92.1%	97.7%

上述线性攻击只能算出一个比特。下面的算法可以给出更多比特。

3.3 线性密码分析原理

线性密码分析是已知明文分析。**原理：**

$$X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] = 0 \quad \text{成立概率} p \neq 1/2$$

3.3 线性密码分析原理

原理：

$$X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] = 0 \quad \text{成立概率} p \neq 1/2$$

$$\Uparrow$$

Y, K^{N_r+1}的部分比特 $\Leftarrow$ 猜测K^{N_r+1}的部分比特

原理：

$$
\begin{array}{rcl}
X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] & = & 0 \qquad \text{成立概率} p \neq 1/2 \\
\Uparrow & & \\
Y, K^{N_r+1}\text{的部分比特} & \Leftarrow & \text{猜测} K^{N_r+1}\text{的部分比特}
\end{array}
$$

设使得$X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] = 0$成立的明文—密文对数量为$T$。则

- 若正确猜测K^{N_r+1}的部分比特值，则$T/N \approx p$，那么$|T/N - 1/2| > 0$。
- 若错误猜测K^{N_r+1}的部分比特值，则$T/N \approx 1/2$，那么$|T/N - 1/2| \approx 0$。

取使得$|T/N - 1/2|$取最大值的K^{N_r+1}的猜测值为正确值。

3.3 线性密码分析原理

原理：

$$
\begin{aligned}
X[i_1, i_2, \ldots, i_a] \oplus W^1[j_1, j_2, \ldots, j_b] &= K^1[s_1, s_2, \ldots, s_u] && \text{第1轮线性逼近概率} p_1 \\
W^1[j_1, j_2, \ldots, j_b] \oplus W^2[l_1, l_2, \ldots, l_c] &= K^2[t_1, t_2, \ldots, t_v] && \text{第2线性逼近概率} p_2 \\
W^2[l_1, l_2, \ldots, l_c] \oplus W^3[k_1, k_2, \ldots, k_d] &= K^3[g_1, g_2, \ldots, g_w] && \text{第3轮线性逼近概率} p_3 \\
&\vdots \\
W^{N_r-1}[r_1, r_2, \ldots, r_e] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] &= K^{N_r}[f_1, f_2, \ldots, f_h] && \text{第} N_r \text{轮逼近概率} p_{N_r} \\
X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] &=
\end{aligned}
$$

3.3 线性密码分析原理

原理：

$$
\begin{aligned}
X[i_1, i_2, \ldots, i_a] \oplus W^1[j_1, j_2, \ldots, j_b] &= K^1[s_1, s_2, \ldots, s_u] && \text{第1轮线性逼近概率} p_1 \\
\oplus & \\
W^1[j_1, j_2, \ldots, j_b] \oplus W^2[l_1, l_2, \ldots, l_c] &= K^2[t_1, t_2, \ldots, t_v] && \text{第2线性逼近概率} p_2 \\
\oplus & \\
W^2[l_1, l_2, \ldots, l_c] \oplus W^3[k_1, k_2, \ldots, k_d] &= K^3[g_1, g_2, \ldots, g_w] && \text{第3轮线性逼近概率} p_3 \\
\oplus & \\
\vdots & \\
\oplus & \\
W^{N_r-1}[r_1, r_2, \ldots, r_e] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] &= K^{N_r}[f_1, f_2, \ldots, f_h] && \text{第} N_r \text{轮逼近概率} p_{N_r} \\
\Downarrow & \\
X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] &= K^1[s_1 \ldots s_u] \ldots K^{N_r}[f_1 \ldots f_h]
\end{aligned}
$$

原理：

$$
\begin{array}{rcll}
X[i_1, i_2, \ldots, i_a] \oplus W^1[j_1, j_2, \ldots, j_b] & = & K^1[s_1, s_2, \ldots, s_u] & \text{第1轮线性逼近概率} p_1 \\
\oplus & & & \\
W^1[j_1, j_2, \ldots, j_b] \oplus W^2[l_1, l_2, \ldots, l_c] & = & K^2[t_1, t_2, \ldots, t_v] & \text{第2线性逼近概率} p_2 \\
\oplus & & & \\
W^2[l_1, l_2, \ldots, l_c] \oplus W^3[k_1, k_2, \ldots, k_d] & = & K^3[g_1, g_2, \ldots, g_w] & \text{第3轮线性逼近概率} p_3 \\
\oplus & & & \\
\vdots & & & \\
\oplus & & & \\
W^{N_r-1}[r_1, r_2, \ldots, r_e] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] & = & K^{N_r}[f_1, f_2, \ldots, f_h] & \text{第} N_r \text{轮逼近概率} p_{N_r} \\
\Downarrow & & & \text{假定每一轮逼近独立} \\
X[i_1, i_2, \ldots, i_a] \oplus W^{N_r}[z_1, z_2, \ldots, z_f] & = & K^1[s_1 \ldots s_u] \ldots K^{N_r}[f_1 \ldots f_h] & \text{根据堆积引理计算概率} p
\end{array}
$$

单轮的线性逼近概率由非线性的S盒决定，因此线性分析的关键是求S盒的线性逼近。

3.4 差分密码分析

差分密码分析是选择明文分析

3.4 差分密码分析

差分密码分析是选择明文分析

定义：对两个比特串x, x^*，用x'表示它们异或，即$x' = x \oplus x^*$。

3.4 差分密码分析

差分密码分析是选择明文分析

定义：对两个比特串x, x^*，用x'表示它们异或，即$x' = x \oplus x^*$。

差分密码分析原理：

$$x' \xrightarrow{p} w'_{r-1}, \quad p > \frac{1}{2^n}, n\text{为明文分组长}$$

3.4 差分密码分析

差分密码分析是选择明文分析

定义：对两个比特串x, x^*，用x'表示它们异或，即$x' = x \oplus x^*$。

差分密码分析原理：

$$(x, x^* = x \oplus x') \Rightarrow x' \xrightarrow{p} w'_{r-1}, \quad p > \frac{1}{2^n}, n\text{为明文分组长}$$

$$\Uparrow$$

$$(w_{r-1}, w^*_{r-1})\text{的部分值}$$

$$\Uparrow$$

$$\text{猜测}K_r\text{中相关的比特,由最后轮函数的逆}g_r^{-1}$$

$$\Uparrow$$

$$(y, y^*)$$

3.4 差分密码分析

差分密码分析是选择明文分析

定义：对两个比特串x, x^*，用x'表示它们异或，即$x' = x \oplus x^*$。

差分密码分析原理：

$$x' \xrightarrow{q_1} w'_1, w'_1 \xrightarrow{q_2} w'_2, \ldots, w'_{r-2} \xrightarrow{q_{r-1}} w'_{r-1}$$

$$\Downarrow$$

$$(x, x^* = x \oplus x') \Rightarrow x' \xrightarrow{p=q_1 q_2 \ldots q_{r-1}} w'_{r-1}, \quad p > \frac{1}{2^n}, n\text{为明文分组长}$$

$$\Uparrow$$

$$(w_{r-1}, w^*_{r-1})\text{的部分值}$$

$$\Uparrow$$

$$\text{猜测}K_r\text{中相关的比特,由最后轮函数的逆}g_r^{-1}$$

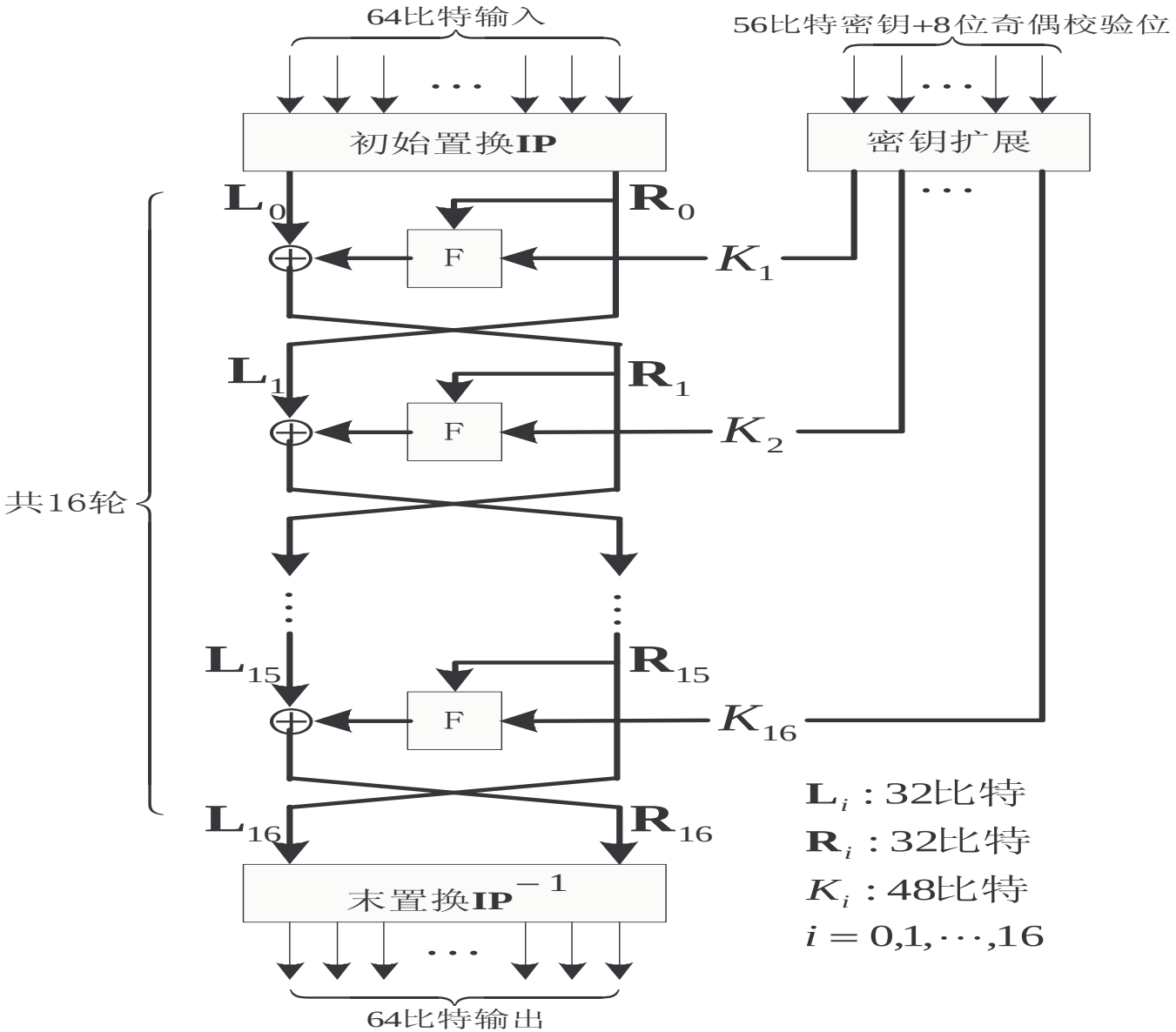
$$\Uparrow$$

$$(y, y^*)$$

3.5 数据加密标准DES
DES的历史

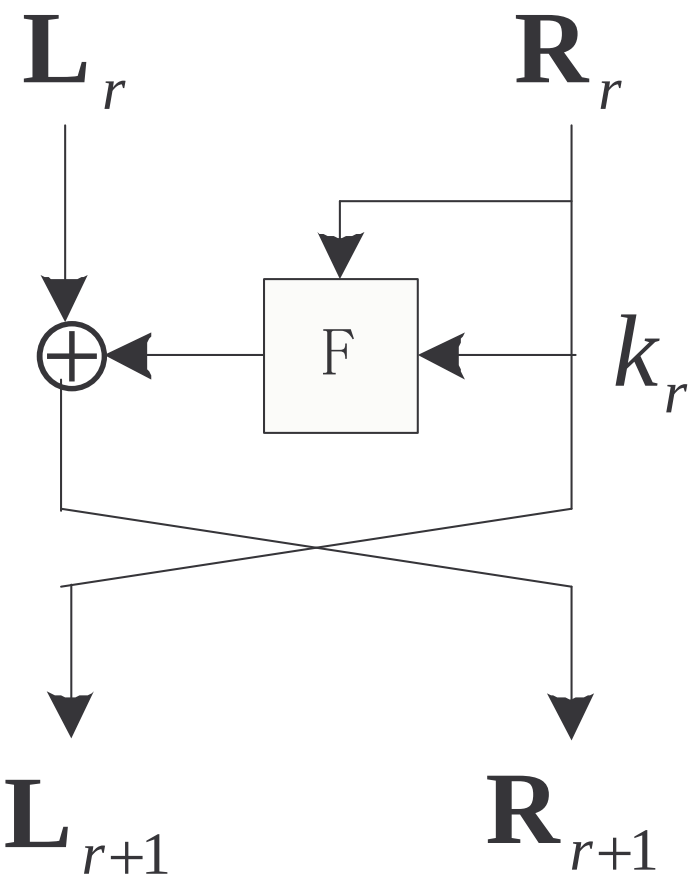
1. 1973年5月15日美国国家技术标准局（NIST）公开征集密码体制。
2. 1977年2月15日由IBM开发的DES被选择为标准，用在“非密级”应用中，DES是对早期版本Lucifer的改进。
3. 每隔5年对DES进行一次评审。
4. 在1999年1日对DES最后一次评审。
5. 2001年11月26日DES最终被AES代替。

DES算法结构图：

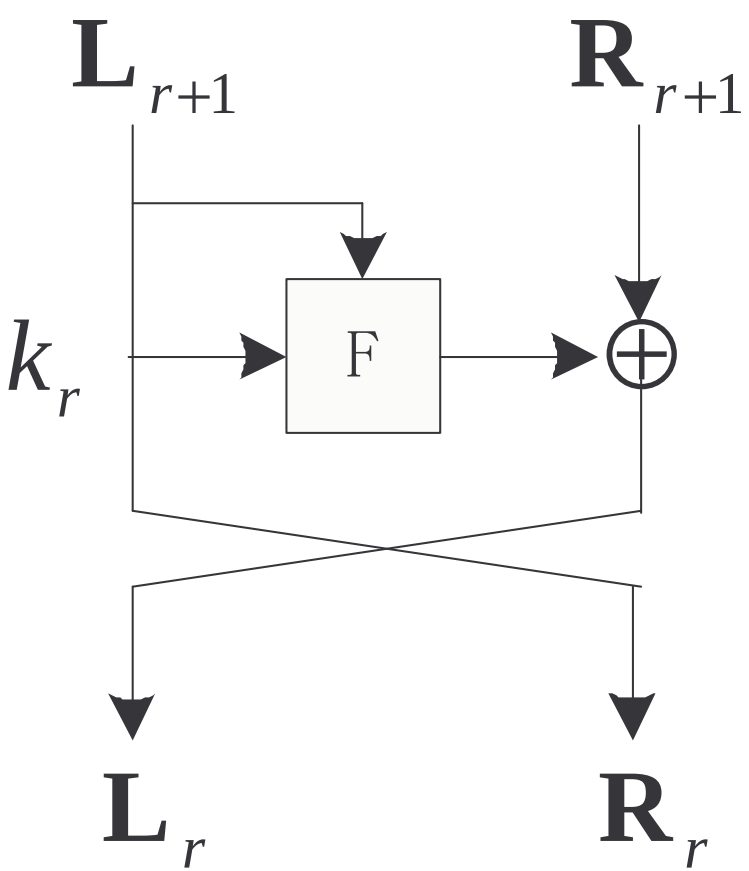

64比特输入
56比特密钥+8位奇偶校验位
初始置换IP
密钥扩展
L_0
R_0
F
K_1
L_1
R_1
F
K_2
共16轮
L_{15}
R_{15}
F
K_{16}
L_{16}
R_{16}
末置换IP^{-1}
64比特输出
L_i：32比特
R_i：32比特
K_i：48比特
$i = 0,1,\cdots,16$

DES的Feistel轮函数结构：

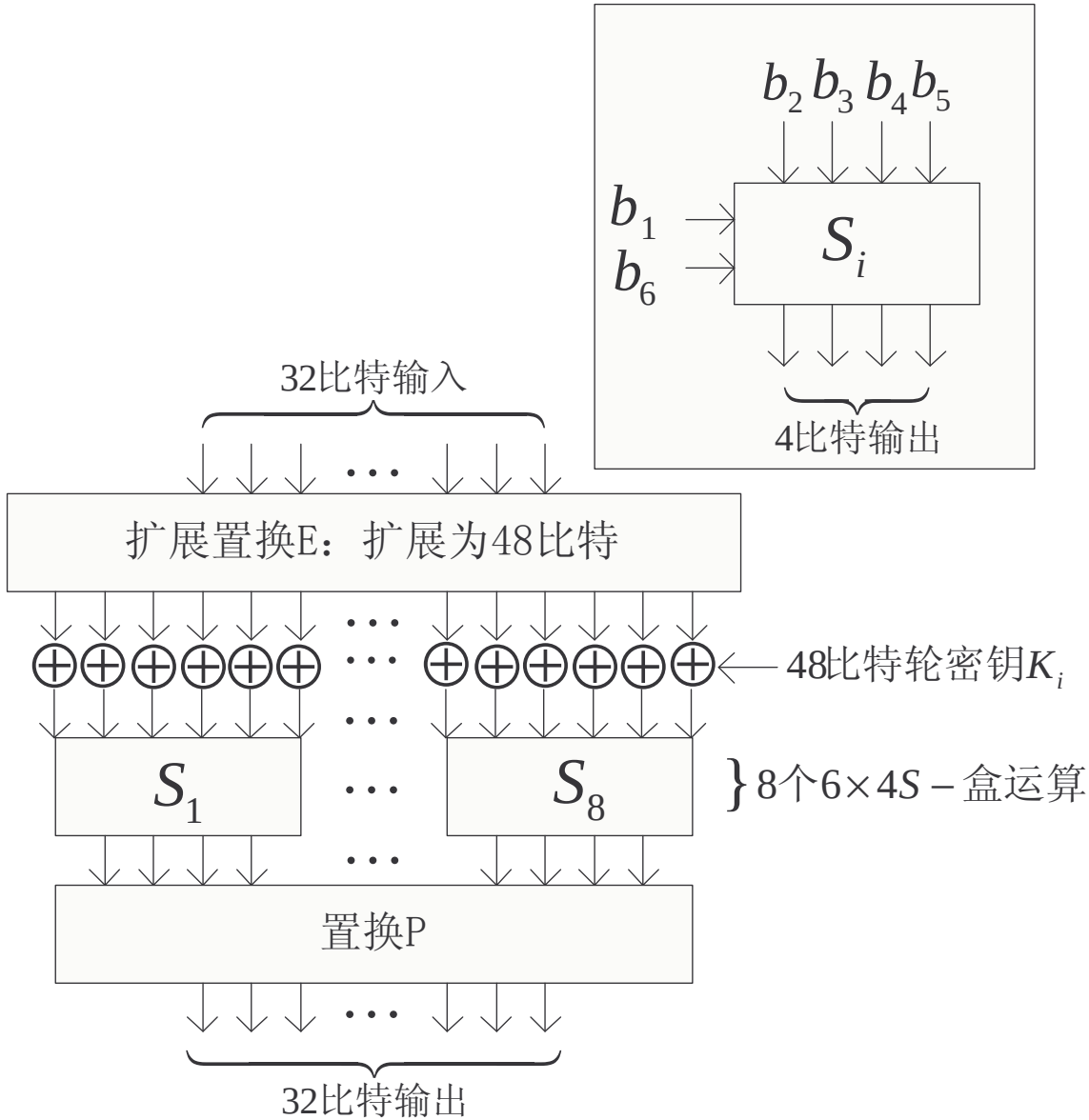
Feistel网络



Feistel网络的
逆过程



*F*函数：


$b_2\ b_3\ b_4\ b_5$
b_1
b_6
S_i
4比特输出
32比特输入
扩展置换E：扩展为48比特
48比特轮密钥K_i
S_1
S_8
}8个6×4S－盒运算
置换P
32比特输出

8个S盒：

S_1															
14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2															
5	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3															
10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
S_4															
7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5															
2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6															
12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7															
4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8															
13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

S盒计算例子：

S_1的输入：$b_1b_2b_3b_4b_5b_6 = (11001)_2$

所在行：$b_1b_6 = (11)_2 = 3$

所在列：$b_2b_3b_4b_5 = (1100)_2 = 12$

输出：$10 = (1010)_2$

	0行	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6

数据加密标准DES
DES算法描述

初始置换IP、末置换IP^{-1}、扩展置换E、置换P、密钥编排算法中的置换PC-1和压缩置换PC-2的运算和古典密码的置换密码一样。

初始置换IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

末置换IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Table: 初始置换IP和末置换IP^{-1}

E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

P			
16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

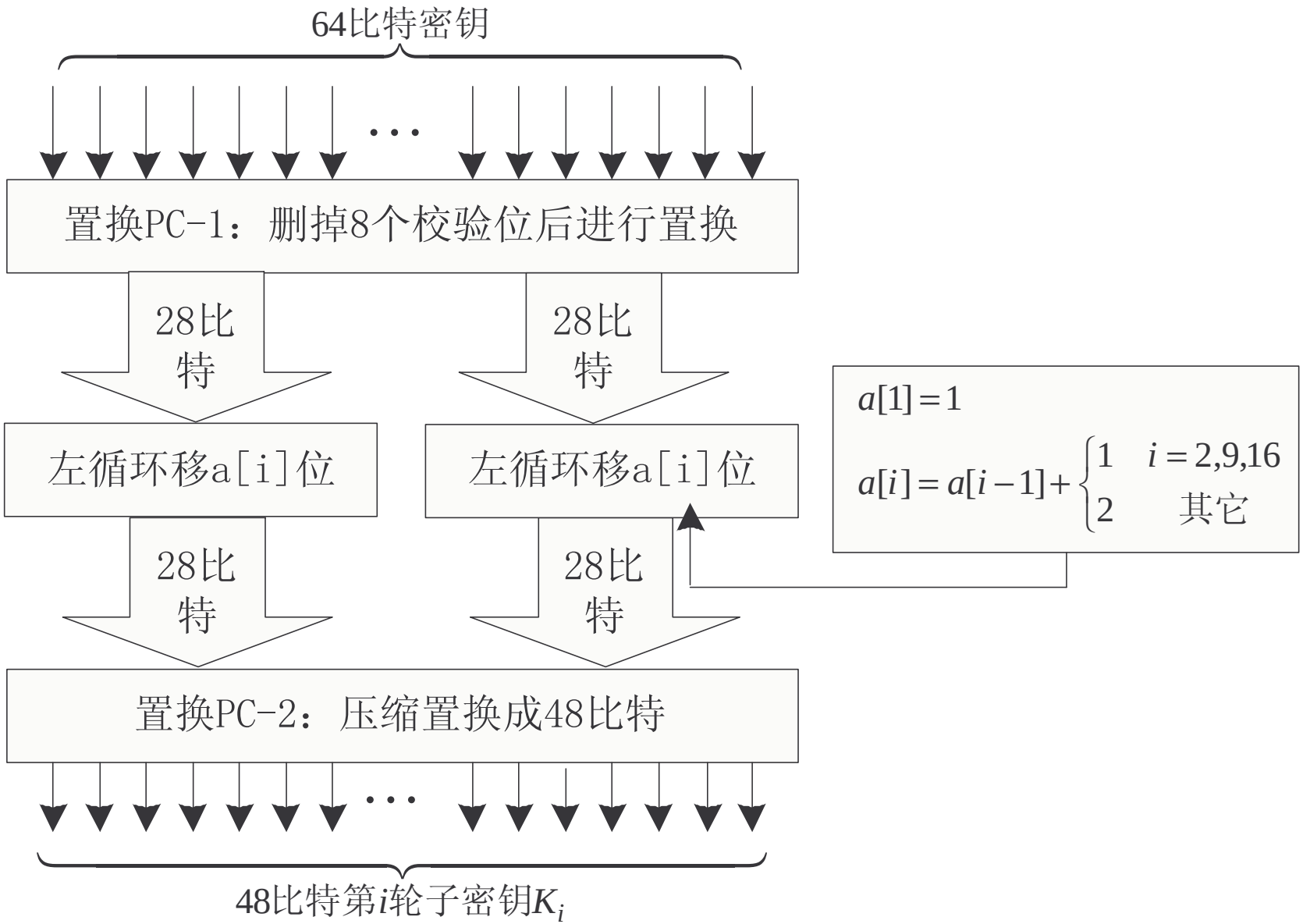
Table: F函数中的扩展置换E和置换P

给定32比特串输入$A = (a_1, a_2, \ldots, a_{32})$,

$$E(A) = (a_{32}, a_1, a_2, a_3, a_4, a_5, a_4, \ldots, a_{31}, a_{32}, a_1)$$

$$P(A) = (a_{16}, a_7, a_{20}, a_{21}, a_{29}, \ldots, a_{11}, a_4, a_{25})$$

密钥编排算法：


64比特密钥
置换PC-1：删掉8个校验位后进行置换
28比特
28比特
左循环移a[i]位
左循环移a[i]位
28比特
28比特
置换PC-2：压缩置换成48比特
48比特第i轮子密钥K_i
$a[1]=1$
$a[i]=a[i-1]+\begin{cases}1 & i=2,9,16\\ 2 & \text{其它}\end{cases}$

PC-1						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

PC-2					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Table: 密钥扩展中的置换PC-1和压缩置换PC-2

数据加密标准DES
DES的分析

对DES的*S*盒的“陷门”争议：

- S-盒是整个DES算法的关键部件，DES靠它实现非线性变换。关于S-盒的设计准则还没有完全公开。
- 许多密码学家怀疑NSA设计S-盒时隐藏了“陷门”，这样只有他们才可以破译算法，但没有证据能标明这点。
- 在1976年，NSA披露了S-盒的几条设计准则。直到1990年，Eli Biham和Adi Shamir提出对DES的差分密码分析后，IBM公布了S-盒和P置换的设计准则。

数据加密标准DES
DES的分析

经过20多年的分析，DES基本上发现没有重大安全缺陷，下表给出了对DES的攻击结果，其中百分比表示成功概率。

分析方法	数据复杂度		存储复杂度	处理复杂度
	已知明文	选择明文		
预处理的穷搜索	–	1	2^{56}	1(查表)
穷搜索	1	–	可忽略	2^{55}
线性攻击	2^{43}(85%)	–	明-密文对量	2^{43}
	2^{38}(10%)	–	明-密文对量	2^{50}
差分攻击	–	2^{47}	明-密文对量	2^{47}
	2^{55}	–	明-密文对量	2^{55}

Table: DES的密码分析结果

预处理的穷搜索：

预处理：

1. 计算对固定明文m的密文表

密文	密钥
$E_{k_0}(m)$	k_0
$E_{k_1}(m)$	k_1
$\vdots$	$\vdots$
$E_{k_{2^{56}-1}}(m)$	$k_{2^{56}-1}$

2. 对密文表的密文列按顺序排序，密钥列进行相应的移动。

密文	密钥
$0\ldots00$	k_{i_0}
$0\ldots01$	k_{i_1}
$\vdots$	$\vdots$
$1\ldots11$	$k_{i_{2^{56}-1}}$

密码分析：

- 输入：m的密文c
- 计算：查找c在排序后的密文表中的位置，输出所对应的密钥。

数据加密标准DES
DES的分析

- 对DES最有效的攻击还是强力攻击，因为无论对差分密码分析，还是线性密码分析，所需要的海量选择明文-密文对是不现实的，而且单是存贮明文-密文对就至少需要140,000GB。
- 事实上，对DES的穷搜索攻击是威胁DES安全的重要因素。例如：
  1. 美国克罗多州的程序员Verser从1997年3月13日起，用了96天的时间，在Internet上数万名志愿者的协同工作下，于6月17日成功地找到了美国RSA公司悬赏一万美金破译的密钥长度为56比特的DES的密钥。
  2. 1998年电子边境基金会(EFF)使用一台25万美元的电脑在56小时内再次破解了56比特的DES。
  3. 1999年1月RSA数据安全会议期间，EFF用22小时15分钟就宣告成功破解DES。但如果DES的密钥长度为128比特，那么它可以在1018年内攻破。(http://www.rsa.com/rsalabs/node.asp?id=2108)

3.5节数据加密标准
DES分析

在新的数据加密标准没有出来之前，增加安全性的方法是对DES进行多重加密：

- 3DES加密：

$$c = E_{k_3}(D_{k_2}(E_{k_1}(m)))$$

解密：

$$m = D_{k_1}(E_{k_2}(D_{k_3}(c)))$$

- 3DES密钥长度为168。
- 如果选取$k_3 = k_1$，密钥长度为112。

为什么不对DES进行2次加密提高安全性呢？

2DES：

$$c = E_{k_2}(E_{k_1}(m)) \Longrightarrow E_{k_1}(m) = D_{k_2}(c)$$

2DES的分析

- 输入：明文密文对$(m_i, c_i), i = 1, \ldots, j$
- 1.构造表1

$E_{k_1}(m_i), i = 1 \ldots j$	k_1
$E_0(m_i), i = 1 \ldots j$	0
$E_1(m_i), i = 1 \ldots j$	1
$\vdots$	$\vdots$
$E_{2^{56}-1}(m_i), i = 1 \ldots j$	$2^{56} - 1$

- 2.构造表2

$D_{k_2}(c_i), i = 1 \ldots j$	k_2
$D_0(c_i), i = 1 \ldots j$	0
$D_1(c_i), i = 1 \ldots j$	1
$\vdots$	$\vdots$
$D_{2^{56}-1}(c_i), i = 1 \ldots j$	$2^{56} - 1$

- 3.查找表1和表2中使得$E_{k_1}(m_i) = D_{k_2}(c_i), i = 1, \ldots, j$成立的对应密钥$k_1, k_2$（可合并两表后排序再查找）。输出$k_1, k_2$。

3.5节数据加密标准
DES分析

2DES分析的复杂度分析：

- 由Shannon理论，2DES的唯一解距离

$$n_0 \approx \frac{\mathrm{lb}|\mathcal{K}|}{R_L \mathrm{lb}|\mathcal{P}|} = \frac{112}{64 R_L}$$

  在这里明文是确定的，明文冗余度$R_L = 1$，则$n_0 \approx 1.75$。
- 取$j = 2$，可以较大概率唯一确定出密钥。此时计算复杂度：
  - 表1构造：$O(2 \cdot 2^{56})$
  - 表2构造：$O(2 \cdot 2^{56})$
  - 查找k_1, k_2：$O(2 \cdot 57 \cdot 2^{57})$

3.6 高级数据加密标准AES

AES的遴选：

1. 1997年1月，美国国家标准技术研究所(NIST)开始对高级数据加密标准（AES）进行研究，并成立了AES标准工作室。
2. 1997年9月12日，NIST发布征集算法的正式公告。
3. 至1998年6月15日提交了21个侯选算法。
4. 于1998年8月20日召开了第一次AES候选人会议（AES1），宣布了来自全世界的15个候选算法。
5. 1999年3月开始的第二次候选人会议，选出了五个决赛算法：MARS, RC6, Rijndael, Serpent和Twofish。
6. 2000年4月召开了第三次AES会议(AES3)。
7. 于2000年10月2日美国商业部长Norman Y.Mineta宣布Rijndeal最终获胜。
8. 2001年11月26日，NIST发布了联邦信息处理标准，即FIPS-197，正式公告了高级数据加密标准(AES)，2002年5月26日公告正式生效。
9. 每隔5年重新进行一次正式评估。

AES的遴选标准

1. AES是公开；
2. AES是分组加密单钥体制，支持128比特长分组和128-，192-和256比特长密钥；
3. AES的密钥长度可变，可以根据需要增加；
4. AES可以用软件和硬件实现；
5. AES可以自由使用，或依据符合美国国家标准技术研究所（NIST）策略的条件使用；

AES的遴选标准

满足以上要求的AES，需要依据以下特性判断优劣：

1. 安全性；至少考虑到以下因素：
   - 与其他侯选算法的实际安全性的比较；
   - 算法对输入分组的输出与随机置换对输入分组的输出的差异程度；
   - 算法安全性所依赖的数学基础是否坚固；
   - 在评估过程中由公众提出的其他安全因素，包括那些证明算法的安全强度比提交者所声明的要弱的任何攻击。
2. 计算效率（从运行速度及存储空间方面考虑）；
3. 灵活性；
4. 使用简单性；
5. 其它在活动中提出的意见。

AES－Rijndeal简介

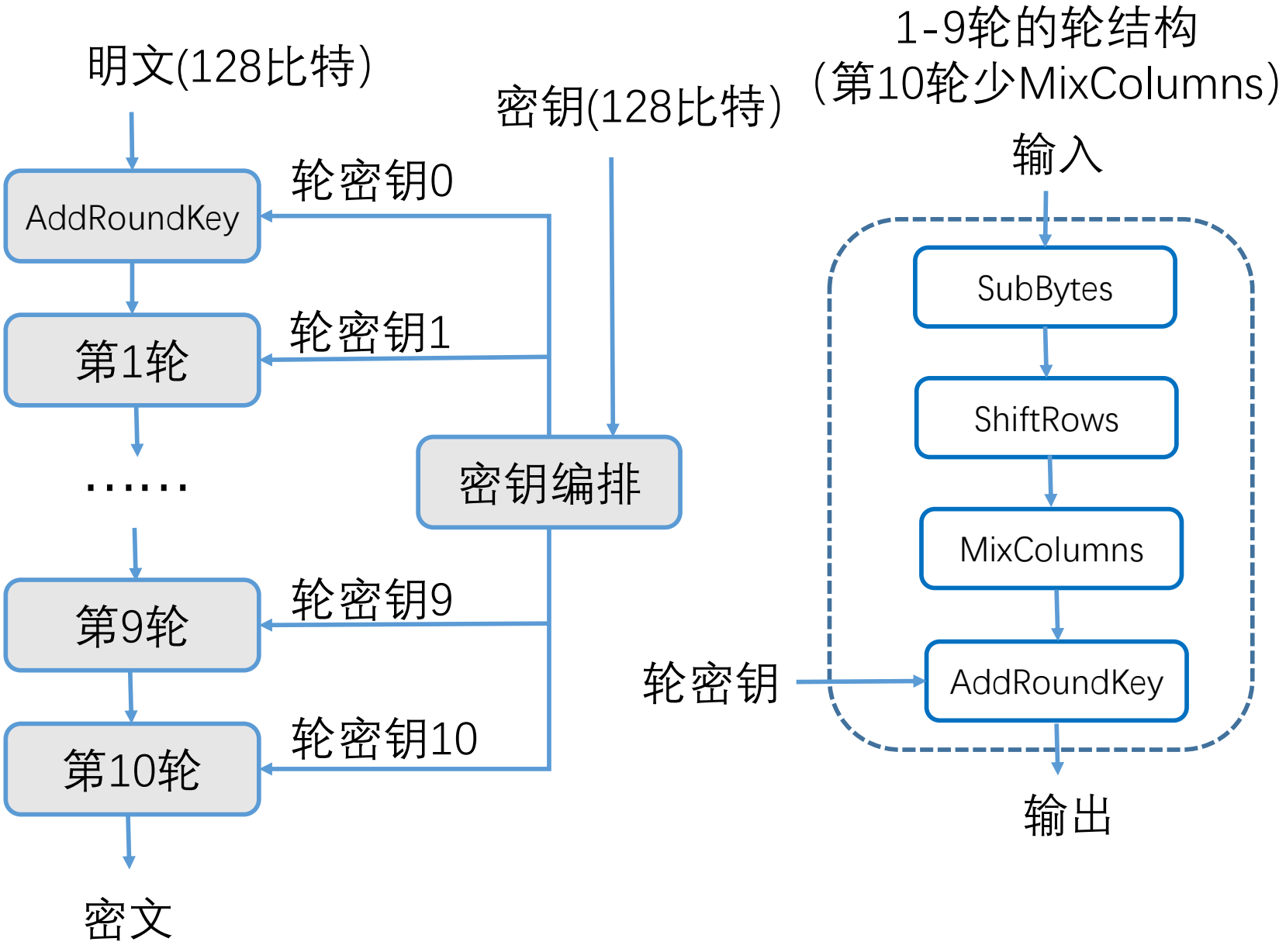
- Rijndeal为比利时密码学家Joan Daemen和Vincent Rijmen所设计(Rijndeal就由两位作者名字的组合而成：Rijmen+Daemen)
- Rijndeal的前身是Square，于1997年在快速软件加密会议(FSE'97)上公布，1998年6月，在对Square作修改、重新命名为Rijndeal后作为AES候选算法。
- Rijndeal胜出的原因：
  - 其软件和硬件实现对计算环境的适应性强，性能稳定、优良。
  - 密钥建立时间短，密钥灵活性好。
  - 而对存储量要求低使它适合资源紧缺的环境，且保持优秀的性能。
  - Rijndeal的运算是所有提交算法中最易于抵抗能量和计时攻击。
  - Rijndeal可灵活组合不同的分组长和密钥长，对算法所作的改动仅是增加它的轮数。

AES的描述

分组长	128	128	128
密钥长	128	192	256
轮数Nr	10	12	14

AES的算法结构：观看128比特密钥的AES的Flash演示。

AES的描述


明文(128比特)
密钥(128比特)
AddRoundKey
轮密钥0
第1轮
轮密钥1
.....
密钥编排
第9轮
轮密钥9
第10轮
轮密钥10
密文
1-9轮的轮结构
(第10轮少MixColumns)
输入
SubBytes
ShiftRows
MixColumns
轮密钥
AddRoundKey
输出

AES的描述
AES的四个运算：

- S盒运算：SubByte
- 行移位：ShiftRow
- 列混合：MixColumn
- 轮密钥异或：AddRoundKey

S盒运算：SubByte

AES把4 × 4个字节表示成的阵列称为状态。

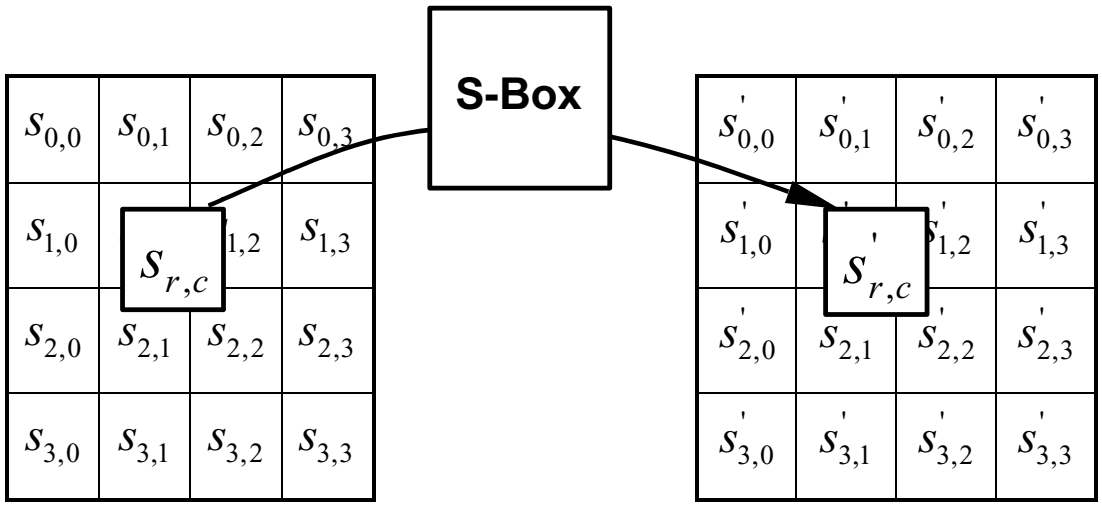


Figure: `SubByte()`作用在状态的单个字节上

S盒运算：SubByte

Subbytes算法$(a_7a_6a_5a_4a_3a_2a_1a_0)$

1. 把比特串$(a_7a_6a_5a_4a_3a_2a_1a_0)$转成有限域$\mathbb{F}_{2^8}$的元素表示$z$。
2. 如果$z \neq 0$，则$z \leftarrow z^{-1}$。
3. 把有限域元素z转成比特串$(a_7a_6a_5a_4a_3a_2a_1a_0)$。
4. $(c_7c_6c_5c_4c_3c_2c_1c_0) = (01100011)$
5. for $i \leftarrow 0$ to 7 do

$$b_i \leftarrow (a_i + a_{i+4} + a_{i+5} + a_{i+6} + a_{i+7} + c_i) \bmod 2$$

   循环中的下标都要经过模8约简。
6. 输出：$(b_7b_6b_5b_4b_3b_2b_1b_0)$

S盒运算：SubByte

Subbytes可以用查表方式实现：

X	Y				
	0	...	B	...	F
0	63	...	2B	...	76
1	CA	...	AF	...	C0
2	B7	...	F1	...	15
3	04	...	E2	...	75
⋮	...	...	...	...	
F	8C	...	0F	...	16

查表方法：

输入 XY，其中X, Y都是4比特

输出 S盒的第X行第Y列的项。

例：

输入 $(00111011)_2$，那么

$$X = (0011)_2 = 0x3, Y = (1011)_2 = 0xB$$

输出 S盒的3行B列的项是$E2$，二进制表示为11100010

行移位：ShiftRow

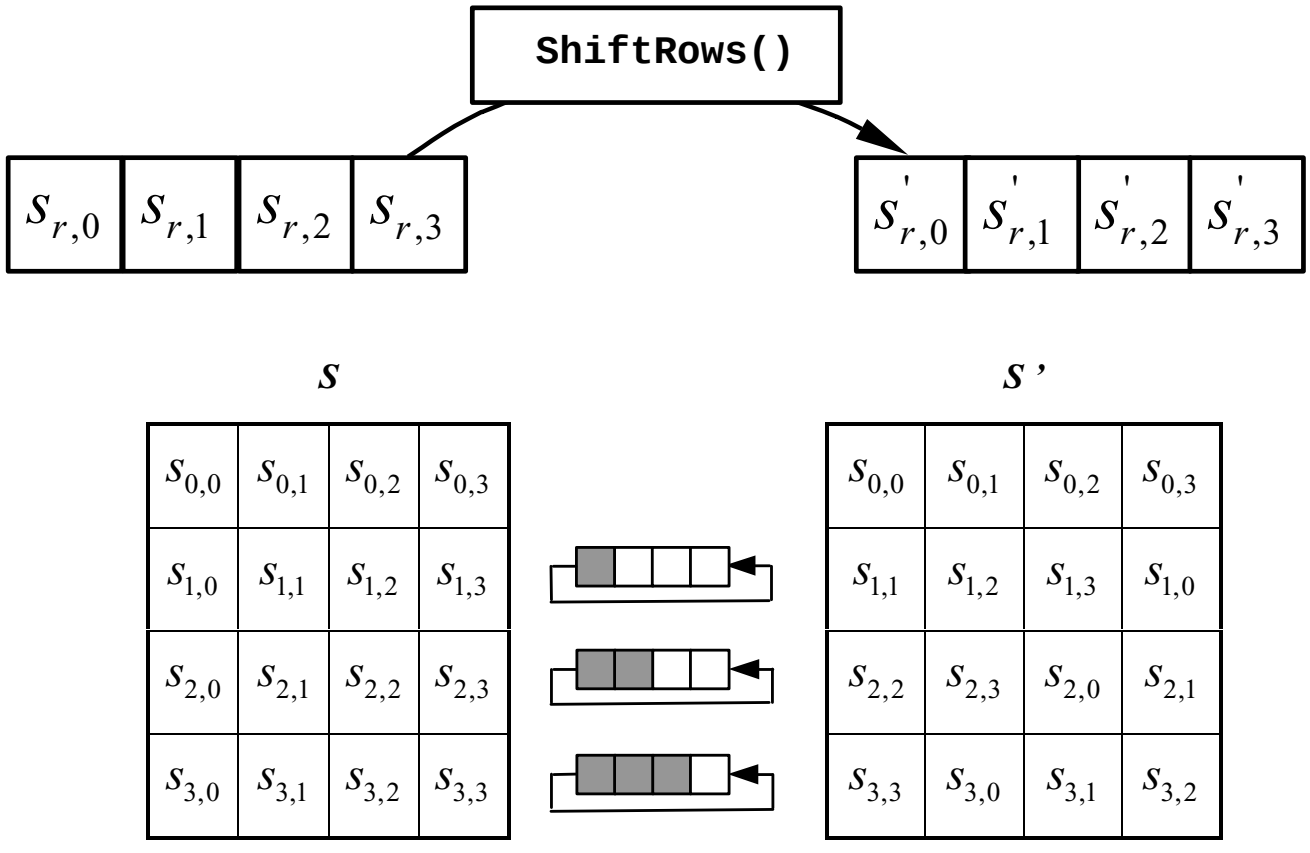


Figure: ShiftRow()作用在状态的行上

列混合：MixColumn

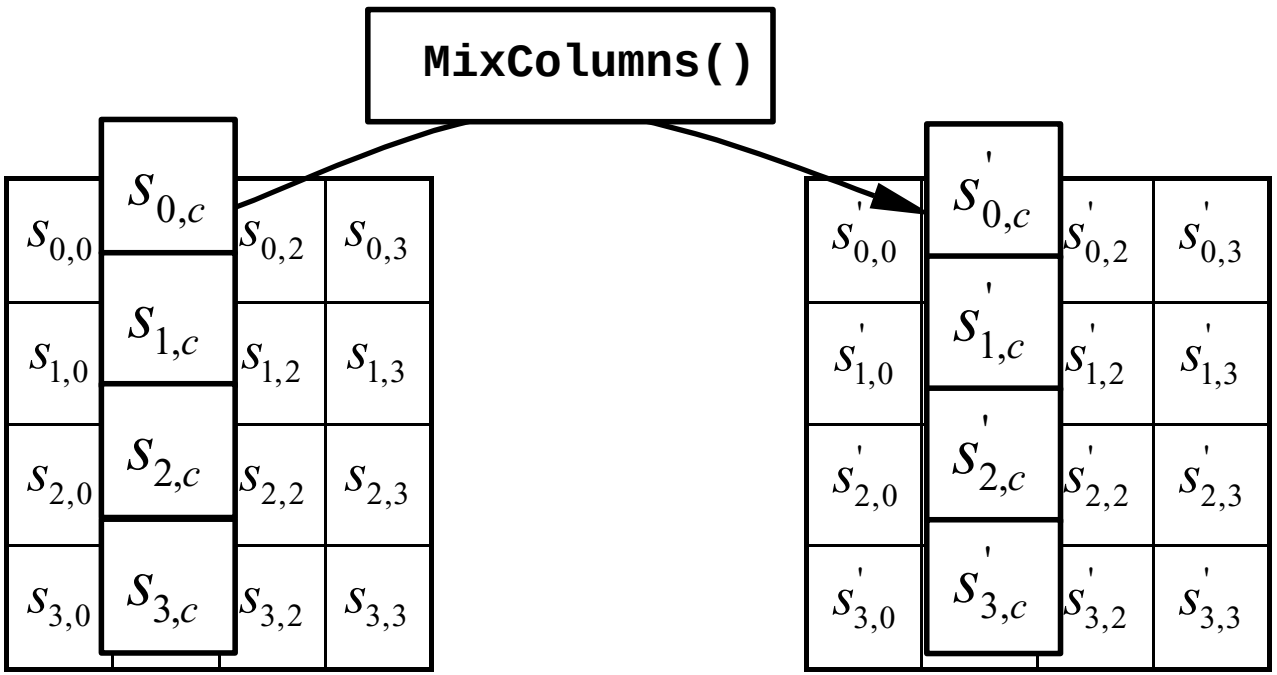


Figure: `MixColumn()`在状态的列上运算

列混合：MixColumn

- MixColumn的数学结构：
  1. 将状态的列视为系数在有限域$GF(2^8)$上的多项式$a(x)$，
  2. 输出$a(x)c(x) \bmod x^4+1$；其中$c(x)$为

$$c(x) = \text{`}03'x^3 + \text{`}01'x^2 + \text{`}01'x + \text{`}02'$$

$c(x)$与x^4+1互素，因此是模x^4+1可逆的。

- MixColumn运算可以表示成$GF(2^8)$上的可逆线性变换

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

课本算法3.5表示了以上的矩阵运算。

列混合：MixColumn

MixColumn(c)的快速软件实现(对第c列的字节$s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c}$操作)

外部算法： $FieldMult(2, x)$

1 for $i \leftarrow 0$ to 3 do $x_i \leftarrow s_{i,c}$

2

$$s_{0,c} = x_1 \oplus x_2 \oplus x_3, \quad s_{1,c} = x_0 \oplus x_2 \oplus x_3$$
$$s_{2,c} = x_0 \oplus x_1 \oplus x_3, \quad s_{3,c} = x_0 \oplus x_1 \oplus x_2$$

3

$$x_0 = FieldMult(2, x_0), \quad x_1 = FieldMult(2, x_1)$$
$$x_2 = FieldMult(2, x_2), \quad x_3 = FieldMult(2, x_3)$$

4

$$s_{0,c} = s_{0,c} \oplus x_0 \oplus x_1, \quad s_{1,c} = s_{1,c} \oplus x_1 \oplus x_2$$
$$s_{2,c} = s_{2,c} \oplus x_2 \oplus x_3, \quad s_{3,c} = s_{3,c} \oplus x_3 \oplus x_0$$

列混合：MixColumn

$FieldMult(2, x)$：

输入 $x = a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0$，是8比特字节

1. $t = a_7$
2. $y = a_6 a_5 a_4 a_3 a_2 a_1 a_0 0$
3. if $t = 1$ $y = y \oplus 00011011$

输出 y

列混合：MixColumn

MixColumn(c)的逆操作的快速软件实现算法：

1 for $i \leftarrow 0$ to 3 do $x_i \leftarrow s_{i,c}$

2

$$s_{0,c} = x_1 \oplus x_2 \oplus x_3, \quad s_{1,c} = x_0 \oplus x_2 \oplus x_3$$
$$s_{2,c} = x_0 \oplus x_1 \oplus x_3, \quad s_{3,c} = x_0 \oplus x_1 \oplus x_2$$

3

$$x_0 = FieldMult(2, x_0), \quad x_1 = FieldMult(2, x_1)$$
$$x_2 = FieldMult(2, x_2), \quad x_3 = FieldMult(2, x_3)$$

4

$$s_{0,c} = s_{0,c} \oplus x_0 \oplus x_1, \quad s_{1,c} = s_{1,c} \oplus x_1 \oplus x_2$$
$$s_{2,c} = s_{2,c} \oplus x_2 \oplus x_3, \quad s_{3,c} = s_{3,c} \oplus x_3 \oplus x_0$$

列混合：MixColumn

MixColumn(c)的逆操作算法(续)：

5

$$x_0 = FieldMult(2, x_0 \oplus x_2), \quad x_1 = FieldMult(2, x_1 \oplus x_3)$$

6

$$\begin{aligned} s_{0,c} &= s_{0,c} \oplus x_0, \quad s_{1,c} = s_{1,c} \oplus x_1 \\ s_{2,c} &= s_{2,c} \oplus x_0, \quad s_{3,c} = s_{3,c} \oplus x_1 \end{aligned}$$

7 $x_0 = FieldMult(2, x_0 \oplus x_1)$

8

$$\begin{aligned} s_{0,c} &= s_{0,c} \oplus x_0, \quad s_{1,c} = s_{1,c} \oplus x_0 \\ s_{2,c} &= s_{2,c} \oplus x_0, \quad s_{3,c} = s_{3,c} \oplus x_0 \end{aligned}$$

密钥异或：AddRoundKey

- AddRoundKey：轮密钥和输入逐比特异或。

$$\begin{array}{|c|c|c|c|} \hline s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ \hline s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ \hline s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ \hline s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|} \hline k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ \hline k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ \hline k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ \hline k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \\ \hline \end{array}$$

$$= \begin{array}{|c|c|c|c|} \hline s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ \hline s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ \hline s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ \hline s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \\ \hline \end{array}$$

密钥编排方案

算法3.6 KeyExpansion(Key)
external RotWord：字节循环左移
SubWord：对字节进行AES的S盒替换
常数$RCon[1], \ldots, RCon[10]$
for $i \leftarrow 0$ to 3
do $w[i] \leftarrow (key[4i], key[4i+1], key[4i+2], key[4i+3])$
for $i \leftarrow 4$ **to** 43

$$\textbf{do} \begin{cases} temp \leftarrow w[i-1] \\ \text{if } i \equiv 0 (\text{mod} 4) \\ \quad \text{then } temp \leftarrow \text{SubWord}(\text{RotWord}(temp)) \oplus Rcon[i/4] \\ w[i] \leftarrow w[i-4] \oplus temp \end{cases}$$

return $w[0], \ldots, w[43]$
第i轮密钥$= w[4i]||w[4i+1]||w[4i+2]||w[4i+3], i = 0, 1 \ldots, 10$

AES的解密函数

1. 密钥编排算法不变，但轮密钥顺序与加密函数的轮密钥相反
2. 所有操作逆序进行
3. 所有操作均为加密函数的逆操作：
   - SubBytes、ShiftRows和AddRoundKey的逆操作容易给出
   - MixColumn的逆操作已提供

AES的分析

- AES的设计能抵抗线性和差分分析
  - S盒使得的线性逼近和差分分布表趋于均匀；
  - 列混合使用了宽轨道策略，使得找到包含较少活动S盒的差分和线性分析成为不可能。
- 目前对AES的最好的分析方法是相关密钥分析：

分组长度	256	192	128
攻击复杂度	$2^{99.5}$	2^{176}	不比穷搜索快

相关密钥分析：假设攻击者能够获得一系列密钥加密的明文和密文对，虽然他不知道密钥值，但是知道这些密钥间的数学关系。

3.7节：工作模式

工作模式：在分组密码中，使用同一密钥多次加密的方法。

1. 一般先对明文进行数据填充，使之长度为分组长度的倍数。
2. 对明文分块，按照某一工作模式加密。

3.7节：工作模式

DES的四种工作模式(1981年加入标准)：

- 电码本模式（ECB模式）
- 密码分组链接模式（CBC模式）
- 输出反馈模式（OFB模式）
- 密码反馈模式（CFB模式）

AES的推荐工作模式：

- 电码本模式（ECB模式）
- 密码分组链接模式（CBC模式）
- 输出反馈模式（OFB模式）
- 密码反馈模式（CFB模式）
- 计数模式（CTR）(2001年加入标准)
- XTS模式(2010年加入标准)

以上工作模式均用于机密性，另有用于消息认证码的工作模式：

- 计数密码分组链接模式（CBC-MAC）：第4章的4.4.2节

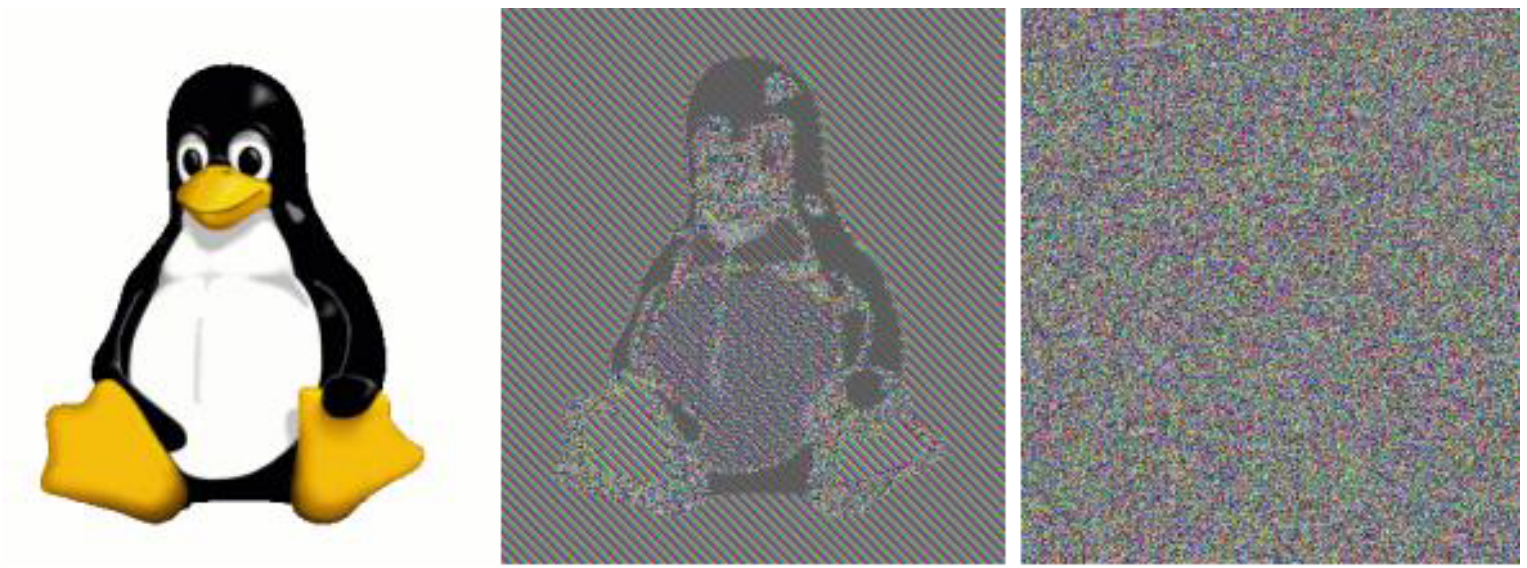
电码本模式（ECB模式）

明文分组序列 $x_1, x_2, \ldots$

加密 $y_1 = e_K(x_1), y_2 = e_K(x_2), \ldots$

解密 $x_1 = d_K(y_1), x_2 = d_K(y_2), \ldots$

- 缺点：同一明文分组加密成同一密文分组，无法完全掩盖明文的统计特性。
  - 例：下图中，以每一像素为分组进行ECB加密，我们看到加密后仍能看出图案的形状，而其它工作模式是看不来的。



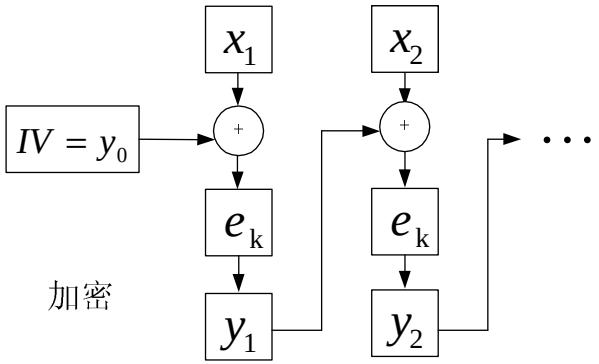
原始图片　　ECB加密模式　　其它加密模式

密码分组链接模式（CBC模式）

初始向量：IV

加密：

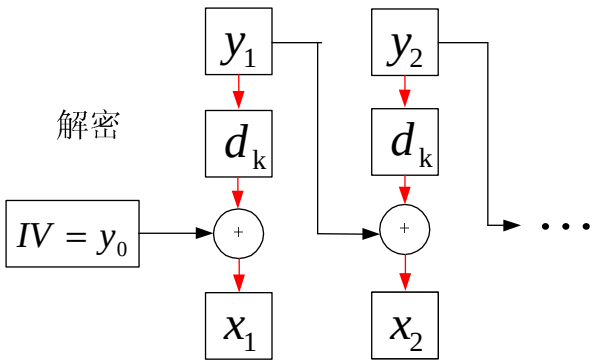
$$
\begin{aligned}
y_0 &= IV \\
y_1 &= e_K(y_0 \oplus x_1) \\
y_2 &= e_K(y_1 \oplus x_2) \\
&\cdots
\end{aligned}
$$



密文：$y_1, y_2, \cdots$

解密：

$$
\begin{aligned}
y_0 &= IV \\
x_1 &= d_K(y_1) \oplus y_0 \\
x_2 &= d_K(y_2) \oplus y_1 \\
&\cdots
\end{aligned}
$$



输出反馈模式（OFB模式）

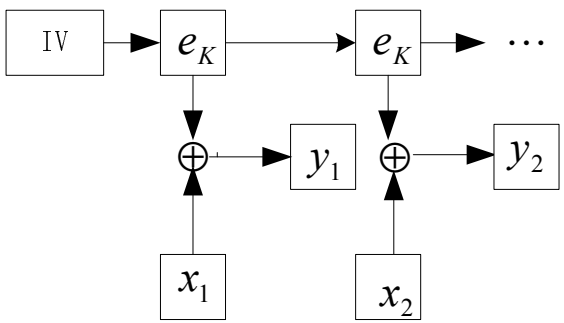
初始向量：IV

加密：

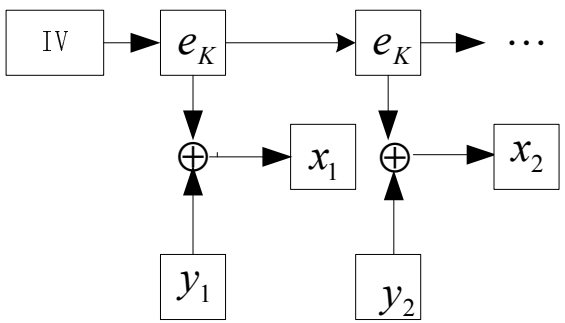
$$
\begin{aligned}
z_0 &= IV \\
z_i &= e_K(z_{i-1}) \\
y_i &= x_i \oplus z_i, i = 1, 2, \ldots
\end{aligned}
$$

解密：

$$
\begin{aligned}
z_0 &= IV \\
z_i &= e_K(z_{i-1}) \\
x_i &= y_i \oplus z_i, i = 1, 2, \ldots
\end{aligned}
$$



OFB加密



OFB解密

输出反馈模式（OFB模式）

每次加密需要更换*IV*：

- 密钥相同，*IV*值如果不变，OFB模数每次加密得到相同的$z_1, z_2, \ldots$，则两次加密的密文的异或＝明文的异或。



XOR



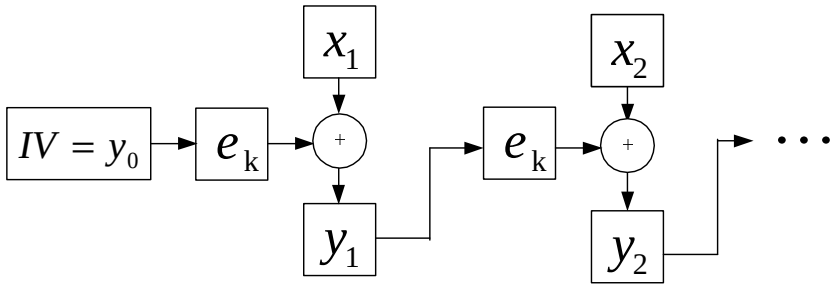


密码反馈模式（CFB模式）

初始向量：IV

加密：

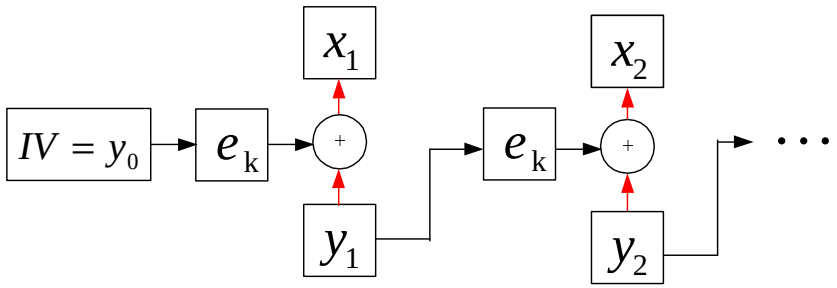
$$
\begin{aligned}
y_0 &= IV \\
z_i &= e_K(y_{i-1}) \\
y_i &= x_i \oplus z_i \\
& \quad i = 1, 2, \ldots
\end{aligned}
$$



加密

解密：

$$
\begin{aligned}
y_0 &= IV \\
z_i &= e_K(y_{i-1}) \\
x_i &= y_i \oplus z_i \\
& \quad i = 1, 2, \ldots
\end{aligned}
$$



解密

计数模式（CTR模式）

计数模式类似于OFB模式，差别是密钥流产生机制不同。
计数模式的工作方式：

1. 计数器ctr：m长比特串
2. 由ctr递归构造m长比特串：

$$T_i = (ctr + i - 1) \bmod 2^m$$

3. 对所有的$i \geq 1$，如下加密明文分组$x_1, x_2, \ldots$

$$y_i = x_i \oplus e_k(T_i)$$

四种工作模式的优缺点

ECB的缺点：相同的明文分组产生相同的密文分组

改变一个明文分组，对四种模式的影响：

- ECB：只影响当前分组
- CBC：当前分组和后续分组都受影响（该特点可用做认证码）
- OFB：只影响当前分组（该特点可用在卫星通信中）
- CFB：当前分组和后续分组都受影响（该特点可用做认证码）

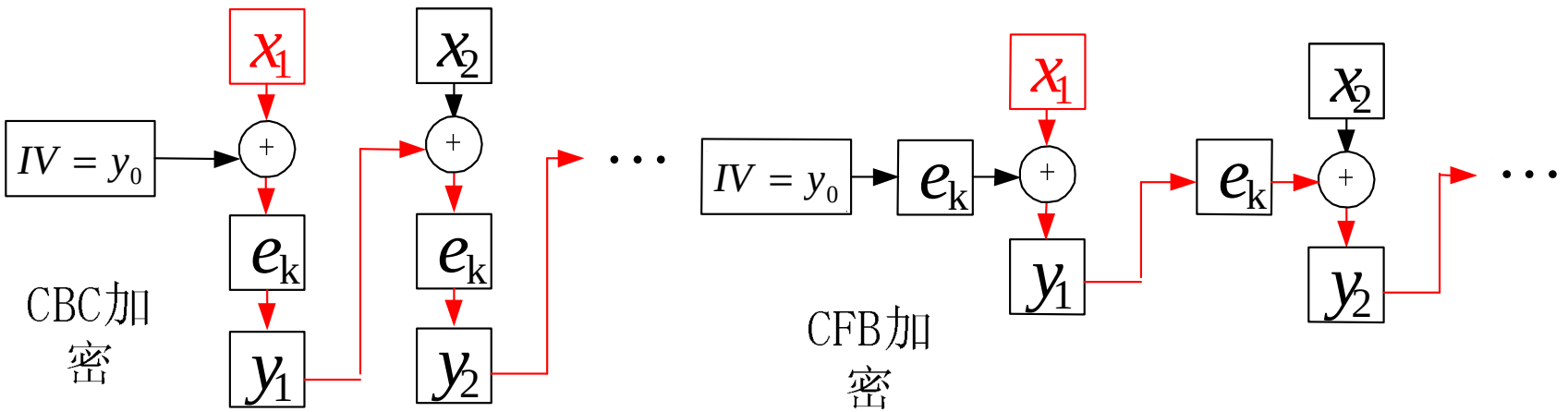


Figure: 一个明文分组的错误对密文的影响

3.7节作业

练习：3.7（讨论一个密文分组出错对解密得到的明文的影响）